

Code Explanation

E2E Address Change YUYU Processing Module Explanation

The E2E Address Change YUYU Processing Module is designed to process address change records from a staging table, create a flat file extract for YUYU output, and update the staging table with processing metadata. This module utilizes Apache Spark for data processing.

Overview of the Code

The code is implemented as a Python class named `AddressChangeProcessor`. It encapsulates the functionality required to read data from a staging table, apply necessary transformations and lookups, prepare the data for YUYU extract and staging update, and finally write the extract to a file and update the staging table.

Step 1: Initialization of the Processor

The `AddressChangeProcessor` class is initialized with a SparkSession and a process user ID. The SparkSession is used for data processing, and the process user ID is stamped on processed records.

```
def __init__(self, spark: SparkSession, process_userid: str):
    self.spark = spark
    self.process_userid = process_userid
    self.current_timestamp = datetime.now()
```

Step 2: Reading Data from the Staging Table

The `read\_staging\_data` method reads data from a specified staging table using the SparkSession.

```
def read_staging_data(self, table_name: str) -> DataFrame:
    return self.spark.table(table_name)
```

Step 3: Applying Lookups to Enrich Data

The `apply\_lookups` method enriches the input DataFrame by applying lookups to retrieve policy owner information from other tables (`T\_YUYU\_CLNT` and `T\_YUYUK\_CLN`).

```
def apply_lookups(self, df: DataFrame) -> DataFrame:
    yuyu_clnt_df = self.spark.table("T_YUYU_CLNT")
    df_with_owner = df.join(yuyu_clnt_df, df["T_TX_BASIC_POLICY_ID"]
    == yuyu_clnt_df["POL_NO"], "left_outer").select(df["*"], yuyu_clnt_
df["POWN_LNM"], yuyu_clnt_df["POWN_FNM"])
    yuyuk_cln_df = self.spark.table("T_YUYUK_CLN")
    result_df = df_with_owner.join(yuyuk_cln_df, df_with_owner["T_TX
_BASIC_POLICY_ID"] == yuyuk_cln_df["POL_NO"], "left_outer").select(d
f_with_owner["*"], yuyuk_cln_df["POWN_KNM"])
    return result_df
```

Step 4: Transforming the Data

The `transform\_data` method applies various business transformations to the enriched DataFrame, including formatting dates, handling null or empty values for postal codes and telephone numbers, and deriving policy owner names.

```
def transform_data(self, df: DataFrame) -> DataFrame:
    return df.withColumn("O__REQUEST_ACCEPT_DATETIME", to_timestamp(
to_date(col("T_TX_REQUEST_REQ_ACC_DATETIME"), "yyyy/MM/dd HH:mm"), "
yyyy-MM-dd HH:mm"))
        .withColumn("v_T_TX_DTL_ADD_CH_N_TRANS_ZIP", when((col
("T_TX_DTL_ADD_CH_N_TRANS_ZIP").isNull()) | (trim(col("T_TX_DTL_ADD_
CH_N_TRANS_ZIP")) == "")) | (length(trim(col("T_TX_DTL_ADD_CH_N_TRANS
_ZIP"))) == 0), col("PPAY_ZIP")).otherwise(col("T_TX_DTL_ADD_CH_N_TR
ANS_ZIP")))
        # Additional transformations...
```

Step 5: Preparing YUYU Extract

The `prepare\_yuyu\_extract` method prepares the DataFrame for YUYU extract by selecting relevant columns and generating a sequence number.

```
def prepare_yuyu_extract(self, df: DataFrame) -> DataFrame:
    return df.withColumn("SEQUENCE_NUMBER", monotonically_increasing
_id()).select(col("SEQUENCE_NUMBER"), col("T_TX_REQUEST_ORIGIN_REQUE
ST_ID").alias("ORIGIN_REQUEST_ID"),
        # Additional column selections...
```

Step 6: Preparing Staging Update

The `prepare\_staging\_update` method prepares the DataFrame for staging update by selecting key identifiers and process stamps.

```
def prepare_staging_update(self, df: DataFrame) -> DataFrame:
    return df.select(col("T_TX_REQUEST_REQUEST_ID"), col("T_TX_BASIC
_TRANS_ID"),
        col("o_STG_PROCESS_DATE").alias("STG_PROCESS_DA
TE"),
        col("o_STG_PROCESS_USERID").alias("STG_PROCESS_
USERID"))
```

Step 7: Writing YUYU Extract to File

The `write\_yuyu\_extract` method writes the prepared YUYU extract DataFrame to a specified output path in CSV format.

```
def write_yuyu_extract(self, df: DataFrame, output_path: str) -> Non
e:
    df.write.mode("overwrite").option("header", "true").csv(output_p
ath)
```

Step 8: Updating the Staging Table

The `update\_staging\_table` method updates the staging table with processing metadata using a MERGE INTO SQL statement.

```

def update_staging_table(self, df: DataFrame, table_name: str) -> None:
    update_view_name = f"temp_update_view_{datetime.now().strftime('%Y%m%d%H%M%S')}"
    df.createOrReplaceTempView(update_view_name)
    self.spark.sql(f"""MERGE INTO {table_name} target USING {update_view_name} source
        ON target.T_TX_REQUEST_REQUEST_ID = source.T_TX_REQUEST_REQUEST_ID
        # Additional join conditions...
        WHEN MATCHED THEN UPDATE SET target.STG_PROCESS_DATE = source.STG_PROCESS_DATE,
        target.STG_PROCESS_USERID = source.STG_PROCESS_USERID""")

```

Step 9: Executing the Full Processing Pipeline

The `process` method orchestrates the entire processing pipeline by calling the above methods in sequence.

```

def process(self, source_table: str, target_table: str, output_path: str) -> None:
    source_df = self.read_staging_data(source_table)
    enriched_df = self.apply_lookups(source_df)
    transformed_df = self.transform_data(enriched_df)
    yuyu_extract_df = self.prepare_yuyu_extract(transformed_df)
    staging_update_df = self.prepare_staging_update(transformed_df)
    self.write_yuyu_extract(yuyu_extract_df, output_path)
    self.update_staging_table(staging_update_df, target_table)

```